

Apache Spark's Basic Architecture

1. Overview of Architecture

- Apache Spark follows a **distributed master-worker architecture** designed for large-scale parallel data processing.
- The architecture consists of a **Driver (master)** and multiple **Worker Nodes**, coordinated through a **Cluster Manager**.
- The system is built to process data by splitting it into smaller chunks (partitions) and executing tasks in parallel across multiple machines.
- Spark primarily works on the concept of **lazy evaluation**, where computations are only executed when an action is triggered.

2. Core Architectural Layers

- **Driver Layer (Control Plane):**
 - Contains the Driver Node, SparkSession, and SparkContext.
 - Responsible for planning, scheduling, and coordinating execution.
- **Execution Layer (Data Plane):**
 - Contains Worker Nodes and Executors.
 - Responsible for actual computation and data processing.
- **Resource Management Layer:**
 - Managed by the Cluster Manager.
 - Allocates CPU, memory, and other resources across the cluster.

3. Detailed Working of Architecture

- **Step 1: Application Submission**
 - A user submits a Spark application using spark-submit or through an interactive environment.
 - The application contains transformations (lazy operations) and actions (trigger execution).
- **Step 2: Driver Initialization**
 - The Driver Node starts and creates a SparkSession.
 - SparkSession internally initializes SparkContext.

- SparkContext establishes communication with the Cluster Manager.
- **Step 3: Resource Allocation**
 - The Cluster Manager receives resource requests from SparkContext.
 - It allocates Worker Nodes and launches Executors on them.
 - Each executor is assigned memory and CPU cores.
- **Step 4: DAG Creation (Logical Plan)**
 - The Driver converts user-defined transformations into a **Directed Acyclic Graph (DAG)**.
 - The DAG represents the sequence of operations and dependencies between them.
 - Since transformations are lazy, no computation happens at this stage.
- **Step 5: DAG Optimization and Stage Division**
 - The DAG is optimized by Spark's scheduler.
 - It is divided into **stages** based on shuffle boundaries (data movement between nodes).
 - Each stage consists of multiple tasks that can be executed in parallel.
- **Step 6: Task Creation and Scheduling**
 - Each stage is split into **tasks**, where each task operates on a single data partition.
 - The Driver schedules these tasks and assigns them to executors.
- **Step 7: Task Execution**
 - Executors on Worker Nodes execute assigned tasks.
 - Data is processed in parallel across partitions.
 - Intermediate results are stored in memory (or disk if needed).
- **Step 8: Shuffle and Data Exchange**
 - If operations require data movement (e.g., groupBy, reduceByKey), a **shuffle** occurs.
 - Data is redistributed across executors based on keys.
 - This is one of the most expensive operations in Spark.
- **Step 9: Result Collection**
 - After execution, results are sent back to the Driver Node.
 - The driver either displays results or writes them to storage systems.

Driver Node

- The Driver Node is the central control unit of a Spark application and runs the user's main program.
- It is responsible for defining the overall workflow and coordinating execution across the cluster.
- **Working:**
 - When a Spark application starts, the driver initializes the SparkSession.
 - It converts user-written transformations and actions into a Directed Acyclic Graph (DAG).
 - The DAG is then divided into stages and tasks.
 - The driver communicates with the cluster manager to allocate resources.
 - It assigns tasks to executors and monitors their execution.
- **Uses:**
 - Controls job execution
 - Maintains metadata about RDDs and DataFrames
 - Handles fault recovery by reassigning failed tasks

SparkSession

- SparkSession is the unified entry point for interacting with Spark and is required to start any Spark application.
- It replaces older contexts such as SQLContext and HiveContext.
- **Working:**
 - When created, it automatically initializes SparkContext.
 - It provides APIs for working with structured and semi-structured data.
 - It acts as a bridge between user code and Spark's execution engine.
- **Uses:**
 - Reading data from sources (CSV, JSON, databases)
 - Performing SQL queries
 - Creating and manipulating DataFrames
 - Managing configurations and runtime settings

SparkContext

- SparkContext is the core engine that connects the application to the cluster manager.
- It is a lower-level abstraction compared to SparkSession.
- **Working:**
 - Establishes communication with the cluster manager.
 - Requests resources (CPU, memory) for executors.
 - Divides data into partitions and distributes them across executors.
 - Manages task execution and scheduling.
- **Uses:**
 - Creating RDDs (Resilient Distributed Datasets)
 - Controlling parallel execution
 - Handling fault tolerance using lineage (recomputing lost data)

Cluster Manager

- The Cluster Manager is responsible for managing resources across the distributed environment.
- It acts as an intermediary between the driver and worker nodes.
- **Working:**
 - Receives resource requests from SparkContext.
 - Allocates worker nodes and launches executors.
 - Monitors resource usage and ensures efficient scheduling.
- **Types:**
 - Standalone: Spark's built-in manager
 - YARN: Used in Hadoop ecosystems
 - Kubernetes: Container-based deployment
 - Mesos: General-purpose cluster manager
- **Uses:**
 - Efficient resource allocation
 - Load balancing
 - Scalability across multiple machines

Worker Node

- Worker Nodes are the machines in the cluster where actual computation takes place.
- Each worker node can host multiple executors depending on available resources.

- **Working:**
 - Receives tasks from the driver via executors.
 - Processes data partitions in parallel.
 - Stores intermediate data in memory or disk.
- **Uses:**
 - Parallel data processing
 - Distributed storage and computation
 - Improving performance through horizontal scaling

Executors

- Executors are processes launched on worker nodes for a specific Spark application.
- Each executor runs independently and handles multiple tasks.
- **Working:**
 - Execute tasks assigned by the driver.
 - Store intermediate results in memory (cache) or disk.
 - Send results back to the driver.
- **Uses:**
 - Speed up computation through parallelism
 - Support in-memory processing for iterative algorithms
 - Enable fault tolerance by re-executing failed tasks

Execution Flow

- The user submits a Spark application.
- The driver program starts and creates SparkSession.
- SparkSession initializes SparkContext.
- SparkContext requests resources from the Cluster Manager.
- Cluster Manager allocates Worker Nodes and launches Executors.
- Driver constructs a DAG based on transformations.
- DAG is divided into stages and tasks.
- Tasks are distributed to executors.
- Executors process data partitions in parallel.
- Results are returned to the driver.

Client Mode

- In Client Mode, the driver runs on the user's local machine.
- Executors run on the cluster.
- **Working:**
 - The client machine submits the job and remains active during execution.
 - The driver directly communicates with executors.
- **Uses:**
 - Development and testing
 - Interactive analysis
- **Advantages:**
 - Easy debugging
 - Direct access to logs
- **Disadvantages:**
 - Application fails if client disconnects
 - Not suitable for long-running jobs

Cluster Mode

- In Cluster Mode, the driver runs inside the cluster.
- Both driver and executors are managed by the cluster manager.
- **Working:**
 - The job is submitted to the cluster manager.
 - The cluster manager launches the driver on a worker node.
 - Executors are then launched to perform tasks.
- **Uses:**
 - Production workloads
 - Long-running jobs
- **Advantages:**
 - High reliability
 - Better fault tolerance
- **Disadvantages:**
 - Debugging is more complex
 - Logs are not directly available on the client

11. Client Mode vs Cluster Mode

- Client Mode: Driver runs on a local machine, suitable for development and debugging.
- Cluster Mode: Driver runs inside cluster, suitable for production and large-scale processing.
- Client Mode depends on client availability, while Cluster Mode is independent of client after submission.
- Cluster Mode provides better resource management and fault tolerance.